

# Embedding Encryption and Machine Learning Intrusion Prevention Systems on Programmable Logic Controllers

Thiago Alves<sup>✉</sup>, Rishabh Das, and Thomas Morris

**Abstract**—During its nascent stages, programmable logic controllers (PLCs) were made robust to sustain tough industrial environments, but little care was taken to raise defenses against potential cyberthreats. The recent interconnectivity of legacy PLCs and supervisory control and data acquisition (SCADA) systems with corporate networks and the Internet has significantly increased the threats to critical infrastructure. To counter these threats, researchers have put their efforts in finding defense mechanisms that can protect the SCADA network and the PLCs. Encryption and intrusion prevention systems (IPSs) have been used by many organizations to protect data and the network against cyber-attacks. However, since PLC vendors do not make available information about their hardware or software, it becomes challenging for researchers to embed security mechanisms into their devices. This letter describes an alternative design using an open source PLC that was modified to encrypt all data it sends over the network, independently of the protocol used. Additionally, a machine learning-based IPS was added to the PLC network stack providing a secure mechanism against network flood attacks like denial of service (DoS). Experimental results indicated that the encryption layer and the IPS increased the security of the link between the PLC and the supervisory software, preventing interception, injection, and DoS attacks.

**Index Terms**—Advanced encryption standard (AES), encryption, intrusion prevention system (IPS) machine learning, OpenPLC, programmable logic controller (PLC), supervisory control and data acquisition (SCADA).

## I. INTRODUCTION

INDUSTRIAL control systems (ICS), also known as supervisory control and data acquisition (SCADA) systems, manage complex and potentially dangerous processes using peripheral devices such as programmable logic controllers (PLC) that directly interface with the process plant or machinery. ICS operational networks can be penetrated by cyber-attackers. Although firewalls, demilitarized zones and data diodes can often isolate ICS operational networks, it cannot be assumed to stop all penetrations.

Manuscript received January 24, 2018; revised March 5, 2018; accepted March 30, 2018. Date of publication April 5, 2018; date of current version September 7, 2018. This work was supported in part by the National Science Foundation under Grant 1623657, and in part by the Office of Naval Research under Grant N00014-17-1-2515. This manuscript was recommended for publication by M. Maniatakos. (Corresponding author: Thiago Alves.)

The authors are with the Department of Electrical and Computer Engineering, University of Alabama in Huntsville, Huntsville, AL 35899-0001 USA (e-mail: thiago.alves@uah.edu; rd0029@uah.edu; tommy.morris@uah.edu).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/LES.2018.2823906

The work presented on this letter makes the assumption that ICS operational networks will be penetrated, and therefore presents an embedded resiliency system that can defeat cyber-attacks after the penetration. Cyber-attacks from three categories are considered: 1) interception; 2) injection; and 3) denial of service (DoS).

Given that vendors do not make available any information about their hardware or software, this letter augmented an open source PLC [1] with advanced encryption standard (AES)-256 encryption and decryption capabilities, and a machine learning-based IPS, to create the embedded resiliency system. The OpenPLC project was created in accordance with the IEC 61131-3 standard [2], which defines the basic software architecture and programming languages for PLCs. This means that OpenPLC can be compatible with other PLCs that are compliant to the same standard.

## II. RELATED WORKS

There are many examples in the literature where researchers applied cryptography to secure SCADA communication. The work put forth by Fovino *et al.* [3] describes a secure version of the Modbus SCADA protocol that incorporates integrity, authentication, nonrepudiation, and anti-replay mechanisms. Since the original Modbus protocol was modified, a secure gateway must be provided to enable legacy devices to communicate.

Majdalawieh *et al.* [4] proposed a security framework for the distributed network protocol version 3 (DNP3). Since DNP3 was never designed with security mechanisms in mind, the protocol lacks any form of authentication or encryption. The described DNPsec architecture adds confidentiality, integrity, and authenticity to the original DNP3 and uses encryption to protect against eavesdropping and to hide the frame source. This solution requires modifications to the DNP3 data link layer and therefore cannot be applied to certified DNP3 devices without a proper gateway to translate the messages.

Although encryption can protect data confidentiality, it cannot protect against network intrusions. Therefore, various algorithms have been proposed to identify network intrusions, where most of them rely on machine learning to identify the attack signatures or the normal pattern of the network.

Some applications of machine learning-based intrusion detection systems (IDSs) for SCADA systems in the literature include Yang *et al.* [5] which proposed an IDS to detect man in the middle (MiTM) and DoS attacks. Zhang *et al.* [6] used a data mining approach to identify malicious data and possible cyber-attacks in communication traffic from different levels of

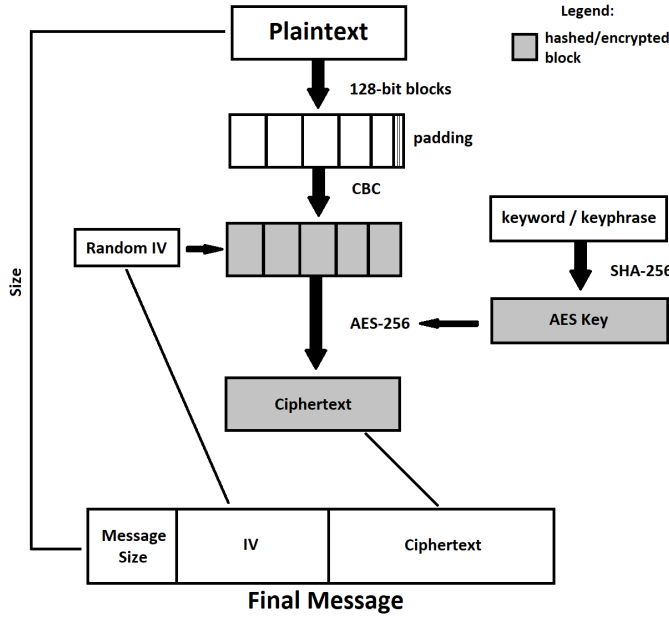


Fig. 1. OpenPLC encryption process.

networks. Hadeli *et al.* [7] proposed anomaly detection techniques which extract behaviors from various communication protocols to create a full description of the communication pattern in an ICS. These approaches work with data strictly from the network and are applying either signature-based or anomaly based IDS.

The work proposed in this letter comes up with a novel architecture of an embedded IPS and encryption for OpenPLC. The embedded architecture improves performance, as each PLC analyzes only its own data, decrease costs, as no physical gateway is needed to translate the messages, and also minimizes the exposure of the IPS to the network.

### III. AES-256 ENCRYPTION MODULE

The AES encryption layer stands between OpenPLC's network layer and the IPS. The key scheme used is preshared key (PSK). Therefore, the user must provide a keyphrase for the AES encryption layer every time OpenPLC is starting. OpenPLC expects all data received to be encrypted. Therefore, every packet received is decrypted by the AES encryption layer using the PSK, and then sent to the network layer for further processing. If a response is needed from the network layer, the reverse process occurs, as the AES encryption layer will encrypt the message using the PSK and then forward the ciphertext to the external network. The encryption process is protocol independent, and therefore works with both Modbus and DNP3 implementations in the original OpenPLC.

The encryption process requires five steps, as illustrated on Fig. 1. The decryption process is analogous to the encryption process illustrated before but executed in reverse order.

Although the AES-256 encryption process can secure messages from OpenPLC, the supervisory software cannot benefit from it if they cannot decrypt the message sent by OpenPLC. For this reason, a localhost gateway application was developed to provide a decryption mechanism inside the machine running the supervisory software. The OpenPLC secure gateway uses the same AES Encryption Layer implemented in OpenPLC. Once started, it prompts the user for

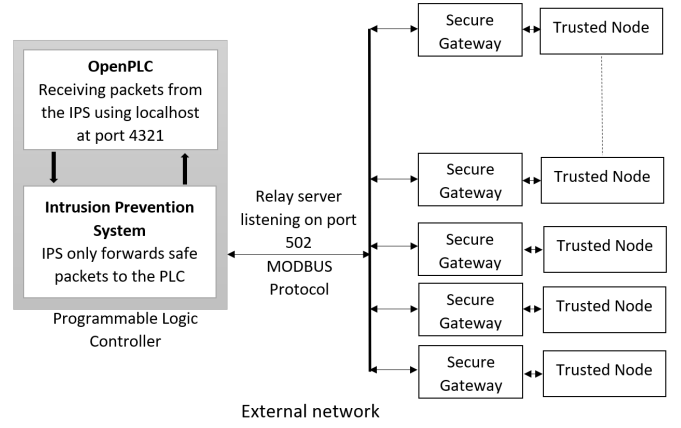


Fig. 2. External network layout of the IPS.

the destination PLC IP Address, the keyphrase to be used for the encryption, and then starts a server listening for Modbus or DNP3 clients. Legacy SCADA software that works with the original OpenPLC should work with the secure OpenPLC, given that the messages go through the gateway.

### IV. EMBEDDED INTRUSION PREVENTION SYSTEM

The IPS interfaces all packets received from the external network. It boasts off an implementation of an embedded unsupervised clustering algorithm to classify the incoming streaming data real-time to detect network anomaly and DoS attacks. If an attack is detected the IPS creates its own custom rules to block the attacker's IP from the network. The unsupervised clustering algorithm (*K*-means) makes the IPS generic and makes it adjustable to any changing network condition that a normal network might experience.

Only the TCP header, packet latency and packet processing information are available to the IPS. After collecting the required data while training or after inspecting the packet while monitoring, the IPS forwards the packet to the PLC. Fig. 2. shows the network layout of the IPS and the trusted nodes connected through the secure gateway.

*K*-means clustering is an unsupervised algorithm to partition  $n$  number of samples into  $K$  number of clusters in which each observation belongs to cluster with the nearest mean. If the set of clusters are represented as  $\{S_1, S_2, S_3, \dots, S_k\}$ . The objective function can be written as

$$\arg \min_s \sum_{i=1}^k \sum_{x \in S_i} \|x - \mu_i\|^2 = \arg \min_s \sum_{i=1}^k |S_i| \text{Var } S_i$$

where  $\mu_i$  is the mean of the points in  $S_i$ .

Once the PLC is commissioned into a new SCADA system the network latency and the processing time of data from different clients are collected real-time by the data collector module of the IPS. After the completion of the data collection, the collected data is clustered into  $n + 1$  different partitions using the *K*-means algorithm, where  $n$  is the number of clients.

### V. EXPERIMENTAL RESULTS

The UAH SCADA Laboratory includes three laboratory scale ICSs: 1) a gas pipeline; 2) a water treatment facility; and 3) a water storage tank. Each system is a working physical model with commercial actuators and sensors. The actuators

and sensors for each system are connected to a Raspberry Pi model 3 attached to a UniPi industrial board running OpenPLC. Each PLC is programmed with ladder logic for distributed control. All three PLCs are networked through a set of Siemens switches to a HMI. The HMI is implemented using the ScadaBR software, which is provided with OpenPLC. The attacks for this letter were performed on the water storage tank.

#### A. Interception Attack

During an interception attack, the attacker monitors data streams to or from a target, in order to gather sensitive information. With a simple packet sniffer, the attacker can read all plaintext traffic that is traveling across the network.

On the water storage tank SCADA system, the information about current mode, defined setpoints and manual triggers for the pump and valve are stored in PLC registers accessed by Modbus or DNP3. Due to the unsecured nature of both protocols, this information can be accessed and modified by anyone in the network.

After starting the modified secure OpenPLC on the same UniPi device, the interception attack was performed once again. Although it was possible to intercept the packets on the network, it was not possible to decypher its contents due to the AES-256 encryption.

#### B. Injection Attack

Injection is an entire class of attacks that rely on injecting data into an application and possibly causing harm by altering the course of execution for the application. The injection attack consisted in quickly sending Modbus messages to lock the system in manual mode, with the pump turned on and the escape valve turned off. Although the HMI was still able to monitor the system, it was not possible to revert the system back to auto mode, or turn off the pump, as messages were being injected faster than the authentic messages from the HMI.

The same attack was performed with the modified secure OpenPLC in place. Due to the fast pace of the injected messages, the embedded IPS detected an anomalous traffic and was able to quickly block the attacker node without causing any harm to the running system. After slowing down the frequency for the injected messages, it was possible to go undetected through the IPS. Although the injected messages were able to reach the PLC, it was still not possible to inject the commands due to the AES-256 encryption that was in place. Therefore, the messages were discarded, and no harm was caused to the system.

#### C. Denial of Service Attack

A DoS attack is a cyber-attack where the perpetrator seeks to make a machine or network resource unavailable to its intended users.

The DoS attack performed for this letter uses the Low Orbit Ion Cannon (LOIC) software to flood the PLC with TCP packets on port 502, which is the same port used for Modbus communications. The attack was able to almost immediately cause a DoS to the HMI, since the OpenPLC running on the UniPi got overwhelmed and could not respond to the HMI requests anymore. Although the HMI got unresponsive, the system continued functioning, running the ladder logic program. However, after about 3 min of continuous attack,

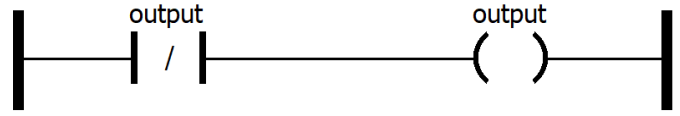


Fig. 3. Ladder logic for the benchmark test.

TABLE I  
SCAN TIME AVERAGE AND STD. DEVIATION

| PLC                 | Average   | Standard Deviation |
|---------------------|-----------|--------------------|
| OpenPLC             | 1.1960 ms | 0.0029 ms          |
| OpenPLC Secure      | 1.1960 ms | 0.0029 ms          |
| Siemens S7-1214C    | 1.3159 ms | 0.2230 ms          |
| A-B MicroLogix 1400 | 0.4579 ms | 0.0549 ms          |

OpenPLC's network stack crashed and the OS killed the application, stopping the ladder logic program execution.

The same attack using LOIC was performed against the secure OpenPLC. Due to the fast nature of the flood packets, the embedded IPS running alongside OpenPLC quickly detected abnormal traffic and banned the attacker node. The attacker node was only able to attack the device for a few milliseconds before being banned.

## VI. BENCHMARK

As demonstrated in the experimental findings from the previous section, the techniques implemented in the open source OpenPLC project indicate that they can thwart MiTM attacks, packet injection attacks, and DoS attacks. However, given the strict real-time requirements of many industrial processes, if the encryption and machine learning IPS layers add significant delays, it might not be possible to deploy these techniques in the real world. Therefore, this section evaluates the performance penalty for applying the encryption and machine learning IPS through a benchmark test.

The test was performed on the original OpenPLC, OpenPLC secure (which is OpenPLC enhanced with the encryption and machine learning IPS layers), and two other popular commercial PLCs: 1) Siemens S7-1214C and 2) Allen-Bradley MicroLogix 1400. Both OpenPLC and OpenPLC secure ran on a Raspberry Pi model 3 hardware. All four PLCs were loaded with the water storage tank ladder logic program.

The real-time behavior of a PLC is usually associated with its scan time. Therefore, the benchmark test consisted in having the water tank program running on each PLC with the extra ladder logic line from Fig. 3, which provides a way to measure the scan time for each PLC. All PLCs were connected to a water tank HMI provided by ScadaBR, that constantly queries the PLCs for data every 100 ms over Modbus/TCP. A micro-controller was wired directly to the processor (or controller) pin of each PLC to measure the output toggle time, collecting 4000 samples from each PLC. The standard deviation and average metrics were calculated for the 4000 samples and are displayed on Table I.

The results indicate that both OpenPLC versions (original and secure) had the same scan time, which was also similar to the scan time of the other two PLCs analyzed. Additionally, the standard deviation for both OpenPLC versions was much smaller than the standard deviation for the other two commercial PLCs, which means that OpenPLC had much less jitter on

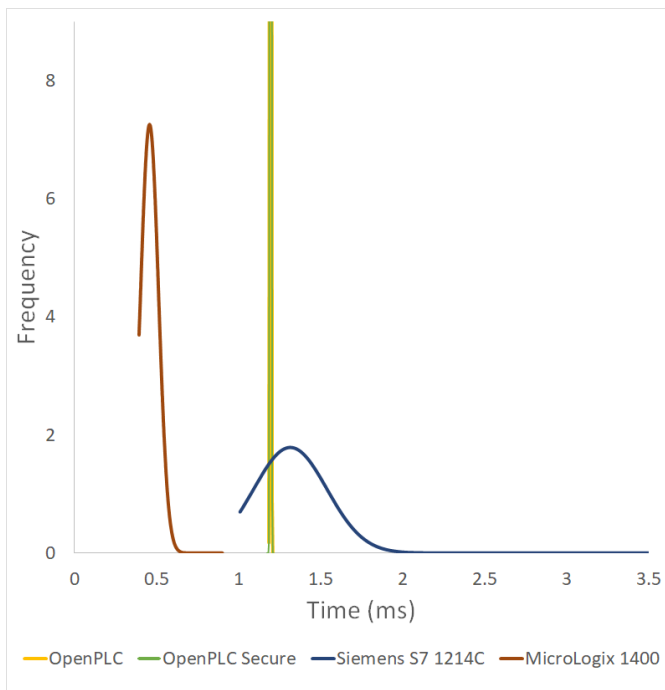


Fig. 4. Histogram for the benchmark test.

the scan time and a much better real-time response, even when the encryption and machine learning IPS layers were added.

The explanation for those results lies in OpenPLC's real-time library, that provides a better real-time response for the OpenPLC process and can really benefit from a multicore architecture such as the one found in the Raspberry Pi model 3. Real-time in Linux is greatly affected by Linux's scheduling policy. When a running process is scheduled out, it is hard to know precisely when it will be scheduled back. This lack of determinism affects processes that require a real-time response. OpenPLC's real-time library improves Linux response on a multicore architecture by isolating OpenPLC process on a separated core, and by changing the scheduling policy for the OpenPLC main thread to SCHED\_FIFO and

setting its priority to the maximum value. Processes under this policy do not have time slicing, which means that a SCHED\_FIFO thread runs indefinitely until it yields the processor or is blocked by an I/O request. Since the OpenPLC process is the only process constantly running in the isolated core, it can provide an improved deterministic response, even when the system is under heavy load.

## VII. CONCLUSION

The combination of the embedded encryption module and the IPS makes the secure version of OpenPLC immune to a wide variety of attacks. Attacks like interception, injection, and DoS, which are the major threats in ICSs, were rendered ineffective by the implemented embedded security measure.

Additionally, as demonstrated in the experimental findings from the previous sections, the techniques implemented in the open source OpenPLC project indicate that they not only can thwart MiTM attacks, packet injection attacks, and DoS attacks, but also that the protection mechanisms did not interfere with the real-time characteristics of OpenPLC.

## REFERENCES

- [1] T. Alves, *The OpenPLC Project*. Accessed: Nov. 6, 2017. [Online]. Available: <http://www.openplcproject.com>
- [2] IEC 61131-3: *Programmable Controllers—Part 3 Programming Languages*, Int. Electrotech. Commission, Geneva, Switzerland, 1993.
- [3] I. N. Fovino, A. Carcano, M. Masera, and A. Trombetta, *Design and Implementation of a Secure Modbus Protocol* (IFIP Advances in Information and Communication Technology). Heidelberg, Germany: Springer, 2009, pp. 83–96.
- [4] M. Majdalawieh, F. Parisi-Presicce, and D. Wijesekera, "DNPSec: Distributed network protocol version 3 (DNP3) security framework," in *Advances in Computer, Information, and Systems Sciences, and Engineering*. Dordrecht, The Netherlands: Springer, pp. 227–234.
- [5] Y. Yang *et al.*, "Intrusion detection system for network security in synchrophasor systems," in *Proc. IET Int. Conf. Inf. Commun. Technol. (IETICT)*, Apr. 2013, pp. 246–252.
- [6] Y. Zhang, L. Wang, W. Sun, R. C. Green, II, and M. Alam, "Distributed intrusion detection system in a multi-layer network architecture of smart grids," *IEEE Trans. Smart Grid*, vol. 2, no. 4, pp. 796–808, Dec. 2011.
- [7] H. Hadeli, R. Schierholz, M. Braendle, and C. Tudu, "Leveraging determinism in industrial control systems for advanced anomaly detection and reliable security configuration," in *Proc. IEEE Conf. Emerg. Technol. Factory Autom. (ETFA)*, Sep. 2009, pp. 1–8.